

Resumen Programación 3

Matias Alvira
matias.alvira@fing.edu.uy

Este resumen fue realizado por un estudiante, puede tener errores, ante la duda siempre ir al libro del curso o consultar a un docente.

Si detectas algún error, o simplemente querés mejorar el documento podés hacerlo accediendo al [proyecto](#) en Overleaf

1 Emparejamiento Estable

1.1 Problema

Tenemos dos conjuntos $M = \{m_1, \dots, m_n\}$ y $W = \{w_1, \dots, w_n\}$ donde el cardinal de ambos es n . Sea $S \subseteq M \times W$ un conjunto de pares ordenados de la forma (m, w) con la propiedad de que cada m y cada w aparecen a lo sumo en un sólo par del conjunto.

Cada m ordena a todos los w 's en una lista de preferencia, donde no hay empates. Los w 's hacen lo mismo con los m 's.

- Decimos que un emparejamiento S es **perfecto** si todos los m 's y los w 's aparecen en un único par.
- Una **inestabilidad** sucede cuando dos pares (m, w) y (m', w') pertenecen a un emparejamiento S , y sin embargo: m prefiere a w' por sobre w y w prefiere a m' por sobre m
- Diremos que S es **estable** si es perfecto y no hay ninguna inestabilidad en el conjunto.

1.2 Algoritmo Gale-Shapley

Gale-Shapley(M, W)

```
1: Inicialmente todos los  $m$  y los  $w$  están libres.
2: while (Existe un  $m$  libre que aún no se le propuso a todos los  $w$ ) do
3:   Elegir un  $m$ .
4:   Sea  $w$  el mejor rankeado en la lista de preferencia de  $m$ , al que  $m$  aún no se le propuso.
5:   if ( $w$  está libre) then
6:     ( $m, w$ ) se emparejan.
7:   else ( $w$  está emparejado actualmente con  $m'$ )
8:     if ( $w$  prefiere a  $m$  antes que a  $m'$ ) then
9:        $m$  permanece libre.
10:    else ( $w$  prefiere a  $m$  antes que a  $m'$ )
11:      ( $m, w$ ) se emparejan.
12:       $m'$  pasa a estar libre.
13:    end if
14:  end if
15: end while
    return conjunto de pares  $S$ .
```

1.2.1 Propiedades del algoritmo

- Todo w se mantiene emparejado desde la primera propuesta que recibe, no vuelve a estar libre.
- La secuencia de parejas de w sólo mejora, en términos de su lista de preferencias.
- La secuencia de w a los que m se propone sólo empeoran, en términos de su lista de preferencia.
- El algoritmo termina luego de a lo sumo n^2 iteraciones, debido a que en cada iteración se realiza una propuesta nueva (**medida de avance**) y sólo hay n^2 propuestas posibles.
- Si m está libre aún no se propuso a todos los w .
- El emparejamiento S resultante es perfecto y también es estable.
- w es una **pareja válida** de m si existe un emparejamiento estable que contenga el par (m, w) .
- w es la **mejor pareja válida** de m si no hay un w' que sea una pareja válida de m y que esté mejor rankeado que w en la lista de preferencias de m ($best(m)$).
- m termina siempre emparejado con $best(m)$.
- Cada w es emparejado con su **peor pareja válida** ($worst(w)$).

En resumen, en el Algoritmo de Gale-Shapley el rol proponente obtiene el mejor emparejamiento posible mientras que quienes reciben las propuestas obtienen el peor emparejamiento posible, esto es interesante porque sucede independiente mente de qué elemento tomemos en el paso (3).

Como sucede esto, *el algoritmo no siempre es capaz de devolver todos los emparejamientos estables posibles.*

1.2.2 Implementación de Gale-Shapley

Si bien el algoritmo a lo sumo realiza n^2 iteraciones en el *while*, para que pueda ejecutarse en orden $O(n^2)$ es necesario utilizar algunas estructuras de datos que nos permitan implementar cada iteración en $O(1)$.

- Utilizamos dos matrices *WomanPref* y *ManPref*, que para la mujer j nos dice quién es el i -ésimo hombre mejor ranqueado ($WomanPref[j, i]$) y que para el hombre j nos dice quién es la i -ésima mujer mejor ranqueada ($ManPref[j, i]$), respectivamente.

Para esto se asume que los conjuntos M y W tienen algún orden en particular.

- Ahora, para poder detectar cuando un hombre está libre utilizamos una lista enlazada simple, donde inicialmente todos los hombres están en dicha lista y cuando consiguen pareja salen de la lista, dicha lista podría funcionar con una política LIFO o FIFO.

De esta forma, construyendo la lista en tiempo $O(n)$, luego durante la iteración podemos identificar qué hombres están libres en tiempo $O(1)$. A su vez, si un hombre pasa a estar libre nuevamente, la lista se actualiza en $O(1)$

- También necesitaremos una forma de identificar para cada hombre, a quién se debe proponer la próxima vez que deba hacer una propuesta. Para esto utilizamos un arreglo *Next*, de forma que la próxima mujer a la que el hombre j -ésimo es $ManPref[j, Next[j]]$.

De esta forma en tiempo $O(1)$ sabemos a quién se debe proponer, y actualizar *Next* también es $O(1)$.

- Necesitaremos una estructura que nos permita identificar si una mujer está libre, o si está emparejada, saber con quién. Para esto se utiliza la estructura *Current*, que en $O(1)$ nos permite consultar lo que queremos y también actualizarlo.
- Por último, necesitamos poder comparar el orden de preferencia de dos hombres para el caso en que un hombre m_i se proponga a la mujer w_k que está emparejada con m_j . La estructura *Ranking*, en la entrada $Ranking[w][m]$ nos indica en qué posición de la lista de preferencias de w se encuentra m .

Construir dicha estructura requiere un tiempo $O(n^2)$ para ser inicializada, pero consultarla es $O(1)$.

Con esta implementación, Gale-Shapley tiene un tiempo de ejecución $\Theta(n^2)$.

2 Análisis de algoritmos

El análisis de algoritmos comprende el estudio acerca de cómo va a evolucionar el tiempo y el espacio en memoria requeridos para resolver un problema en función del tamaño de dicho problema. En esencia, se busca saber qué tan eficiente es un algoritmo para un cierto tamaño de entrada. hola

Para medir cuan eficiente es un algoritmo utilizaremos las notaciones O , Ω , Θ y o . Donde cada una expresa en forma diferente la tasa de crecimiento de los tiempos de ejecución y del espacio en memoria.

Dichas medidas son insensibles a factores constantes y términos de menor orden.

2.1 Notación O

Sea $T(n)$ una función que describe el comportamiento de un cierto algoritmo para un tamaño de entrada n . Y sea $f(n)$ una función, diremos que $T(n) = O(f(n))$ si existen constantes $c > 0$ y $n_0 \geq 0$ tales que $\forall n \geq n_0$ se cumpla que $T(n) \leq c \cdot f(n)$.

Esta notación se utiliza para indicar que el rendimiento de un algoritmo está acotado superiormente por la función f .

2.2 Notación Ω

Sea $T(n)$ una función que describe el comportamiento de un cierto algoritmo para un tamaño de entrada n . Y sea $g(n)$ una función, diremos que $T(n) = \Omega(g(n))$ si existen constantes $c > 0$ y $n_0 \geq 0$ tales que $\forall n \geq n_0$ se cumpla que $T(n) \geq c \cdot g(n)$.

Esta notación se utiliza para indicar que el rendimiento de un algoritmo está acotado inferiormente por la función g .

2.3 Notación Θ

Sea $T(n)$ una función que describe el comportamiento de un cierto algoritmo para un tamaño de entrada n , si podemos mostrar que $T(n) = O(f(n))$ y también que $T(n) = \Omega(f(n))$ entonces significa que encontramos una cota tanto inferior como superior que están a menos de un factor constante, por lo tanto $T(n)$ va a crecer exactamente como un múltiplo de $f(n)$ a partir de cierto n . Esto lo escribiremos como $T(n) = \Theta(f(n))$.

Esta notación se utiliza para indicar que el rendimiento de un algoritmo está acotado superior e inferiormente por la función f .

2.4 Notación o

Sea $T(n)$ una función que describe el comportamiento de un cierto algoritmo para un tamaño de entrada n . Y sea $f(n)$ una función, diremos que $T(n) = o(f(n))$ si se cumple que: $\lim_{x \rightarrow \infty} \frac{T(n)}{f(n)} = 0$, es decir que $f(n)$ es una cota superior holgada de $T(n)$.

También se puede ver como que $T(n) = o(f(n))$ si para cualquier constante $\epsilon > 0$ existe una constante $n_0 \geq 0$ tal que para $n \geq n_0$ se cumple que $T(n) < \epsilon \cdot f(n)$.

2.5 Propiedades

Sean f, g y h funciones, entonces:

- Si $f = O(g)$ y $g = O(h)$, entonces $f = O(h)$. (Transitiva de $O(\cdot)$)

- Si $f = \Omega(g)$ y $g = \Omega(h)$, entonces $f = \Omega(h)$. (Transitiva de $\Omega(\cdot)$)
- Si $f = \Theta(g)$ y $g = \Theta(h)$, entonces $f = \Theta(h)$. (Transitiva de $\Theta(\cdot)$)
- Si $f = O(h)$ y $g = O(h)$ entonces $f + g = O(h)$. (Se puede generalizar para la suma de k funciones)
- Para cada $b > 1$ y cada $x > 0$, se cumple que $\log_b n = O(n^x)$. (La base del logaritmo es indiferente porque se puede pasar de una base a otra multiplicando por una constante $\log_a n = \frac{\log_b n}{\log_b a}$)
- Para cada $r > 1$ y cada $d > 0$, se cumple que $n^d = O(r^n)$. (Cualquier función exponencial de base mayor a 1 crece más rápido asintóticamente que un polinomio)

IMPORTANTE Cuando analizamos un algoritmo, muchas veces sucede que dicho algoritmo contiene instrucciones cuyo tiempo depende de cómo se implemente alguna parte del problema o qué estructuras de datos se utilicen, por eso es importante prestar atención a esos detalles.

3 Grafos

3.1 Conceptos básicos

- Un **grafo** $G = (V, E)$ consiste en una colección V de nodos y una colección E de aristas. Los nodos son la representación de algún estado u objeto de la realidad, mientras que las aristas representan una relación entre dos nodos.
Esto último se representa como $e \in E$ tal que $e = u, v$ siendo $u, v \in V$.
- Decimos que un **grafo es no dirigido** cuando las aristas representan una relación simétrica, es decir, que si existe la arista $e = (u, v)$ también existe la arista $e' = (v, u)$.
- Decimos que un **grafo es dirigido** cuando las aristas representan una relación asimétrica, es decir, que si existe la arista $e = (u, v)$ no necesariamente existe la arista $e' = (v, u)$.
- Definimos un **camino** en un grafo no dirigido $G = (V, E)$ como una secuencia de nodos $v_1, v_2, \dots, v_{k-1}, v_k$ con la propiedad de que cada par consecutivo v_i, v_{i+1} está unido por una arista que va desde v_i hasta v_{i+1} .
- Decimos que es un **camino simple** si todos los vértices son diferentes entre sí.
- Si dado un camino $v_1, v_2, \dots, v_{k-1}, v_k$ donde $k > 2$, los primeros $k - 1$ nodos son distintos entre sí y $v_1 = v_k$, entonces ese camino es un **ciclo**.
- Dado un grafo decimos es **conexo** (o **fuertemente conexo** en el caso de los grafos dirigidos) si todos sus nodos son alcanzables mutuamente, es decir, que existen caminos entre todos los nodos.
- Definimos la **distancia** entre dos nodos u, v como el número mínimo de aristas de los caminos $u - v$. En el caso en que no haya un camino entre ambos nodos la distancia ∞ .
- Un grafo no dirigido es un **árbol** si es conexo, tiene $n - 1$ aristas y no contiene ciclos. Eliminar cualquier arista lo volvería desconexo.
- Un nodo es una **hoja** si no tiene descendientes.

Existen dos formas básicas de representar un grafo, mediante una *matriz de adyacencia* o una *listas de adyacencia*.

Matriz de adyacencia

Dado un grafo $G = (V, E)$ tal que la cantidad de nodos es n y la cantidad de aristas es m , lo representamos como una matriz A de $n \times n$. Entonces si $A = [u, v] = 1$ significa que la arista $(u, v) \in G$ y en caso de que no pertenezca al grafo $A = [u, v] = 0$.

Esta representación cumple lo siguiente:

1. Consultar la existencia de una arista es $O(1)$.
2. La representación ocupa $\Theta(n^2)$ espacio. Es especialmente ineficiente cuando la cantidad de aristas es bastante menor a n^2 .
3. Consultar todas las aristas incidentes a un nodo toma un tiempo $\Theta(n)$ porque es necesario chequear toda la columna asignada a ese nodo, independientemente de la cantidad de aristas incidentes que tenga efectivamente.
4. Si el grafo es no dirigido, la matriz resulta simétrica.

Listas de adyacencia

Dado un grafo $G = (V, E)$ tal que la cantidad de nodos es n y la cantidad de aristas es m , lo representamos como una arreglo de n listas de adyacencia de largo m . Entonces si el nodo v pertenece a la lista u , significa que la arista $(u, v) \in E$.

Esta representación cumple lo siguiente:

1. Consultar la existencia de una arista es $O(n)$.
2. La representación ocupa $\Theta(n + m)$ espacio. Especialmente útil en grafos con pocas aristas.
3. Si el grafo es no dirigido, entonces si tenemos la arista $(u, v) \in E$, como u pertenece a la lista de v y viceversa, se cumple que la suma de los largos de las listas es $2m = O(m)$.

3.2 Conectividad y recorrido de grafos

3.2.1 Breadth-First Search (BFS)

El algoritmo *BFS* dado un grafo G permite recorrer el grafo y también permite determinar la conectividad.

El algoritmo inicia en un nodo s que diremos que pertenece a la capa L_0 , y explora hacia afuera del nodo en todas las direcciones, agregando los nodos adyacentes a él a una misma capa, la capa L_1 .

Luego, el algoritmo continúa tomando los nodos $u \in L_j$ con $j > 0$ y explorando los adyacentes, y si no han sido visitados pasan a formar parte de la capa L_{j+1} .

Una propiedad de los algoritmos de recorridos de grafos es que producen un árbol T cuya raíz es el nodo inicial s , donde en el nivel $i \geq 0$ se encuentran los nodos de la capa L_i .

- Para cada $j \geq 1$, la capa L_j producida por *BFS* consiste en todos los nodos a distancia j del nodo inicial s . Existe un camino desde s hasta t si y sólo si t aparece en alguna capa.
- Sea T el árbol generado por *BFS*, sean x, y nodos de T pertenecientes a las capas L_i y L_j respectivamente y sea (x, y) una arista de G . Entonces i y j difieren a lo sumo en 1.
- El conjunto producido al final del algoritmo es la componente conexa de G conteniendo a s .

BFS(G,s)

```

1: Descubierto[s] = true y Descubierto[v] = false para todo  $v \in V$ ,  $v \neq s$ .
2: L[0] que consiste en un único elemento s.
3: i = 0.
4:  $T_{BFS} = \emptyset$ .
5: while (L[i] no está vacío) do
6:   Creamos una lista vacía L[i + 1].
7:   for (cada nodo  $u \in L[i]$ ) do
8:     for (cada arista  $(u, v)$  incidente a u) do
9:       if (Descubierto[v] = false) then
10:        Descubierto[v] = true
11:        Agregamos la arista  $(u, v)$  al árbol  $T_{BFS}$ 
12:        Agregamos v a la lista L[i + 1]
13:      end if
14:    end for
15:  end for
16:  Incrementamos el contador i.
17: end while
    return  $T_{BFS}$ 

```

3.2.2 Depth-First Search (DFS)

Al igual que el algoritmo *BFS*, el algoritmo *DFS* permite recorrer el grafo y también permite determinar la conectividad.

Dado un grafo G , el algoritmo comienza en un nodo s y se busca explorar a un nodo adyacente v , luego de forma recursiva se realiza el mismo proceso hasta llegar a un nodo al que ya se le han explorado todos sus adyacentes.

Si bien ambos algoritmos visitan el mismo conjunto de nodos, lo hacen en un orden distinto, lo que redundará en que la forma de los árboles generados por cada algoritmo sean muy diferentes entre sí.

El siguiente algoritmo devuelve el conjunto de vértices que pertenecen a la misma componente conexa que un nodo inicial s , en el conjunto de vértices R . Normalmente también se desea saber cuál es el árbol, subgrafo de G , que fue encontrado por el algoritmo, en el caso de DFS, basta con guardar el padre de cada nodo a medida que los vamos explorando.

DFS-Recursivo(G,s)

```

1: Marco s como explorado
2: Agrego s a R. ▷ Opcional: esto permite saber que pertenecen a la misma componente conexa.
3: for cada arista  $(s, v)$  incidente a s do
4:   if v no ha sido explorado then
5:     Padre[v] = s                                     ▷ Opcional: esto permite reconstruir el camino.
6:     Invoco DFS – Recursivo(G, v)
7:   end if
8: end for

```

DFS-Iterativo(G, s)

```
1: Inicializamos el árbol  $T_{DFS} = \emptyset$ .
2: Definimos el arreglo  $Padre[w]$ .
3: Inicializamos la pila  $S$  con el elemento  $s$ .
4:  $Explorado[w] = false \forall w \in V$ .
5: while  $S$  no está vacío do
6:   Tomamos un nodo  $u$  de  $S$ .
7:   if  $Explorado[u] = false$  then
8:      $Explorado[u] = true$ .
9:     for cada arista  $(u, v)$  incidente a  $u$  do
10:      Agregamos  $v$  a la pila  $S$ .
11:      Agregamos el padre de  $v$ ,  $Padre[v] = u$ 
12:    end for
13:   end if
14: end while
    return  $T_{DFS}$ 
```

3.2.3 Implementaciones

Tanto BFS como DFS pueden implementarse en $O(m + n)$, siendo m la cantidad de aristas del grafo y n la cantidad de nodos.

Para realizar la recorrida BFS, se utiliza una cola, de esta forma se recorren los nodos en el orden en que fueron encontrados. Mientras que para implementar DFS, se puede hacer de forma iterativa utilizando una pila, de esta forma se visita primero al último nodo encontrado.

También es necesario utilizar un arreglo, que nos indique si ya fueron explorados. Y en el caso de DFS iterativo, también es necesario utilizar un arreglo Padre que nos indica desde qué nodo x accedimos a y ($Padre[y] = x$).

Podemos ver que ambos se ejecutan en $O(m + n)$ ya que un nodo es agregado a la pila o a la cola, únicamente si anteriormente no fue marcado como explorado. Por lo tanto, como dichas estructuras a lo sumo llegan a tener n elementos y como cada arista es vista por ambos extremos, entonces ambos algoritmos realizan $n + 2m$ operaciones, lo cual es $O(m + n)$.

3.2.4 Aplicaciones de BFS-DFS

- Si un grafo es bipartito, entonces no puede contener un ciclo de largo impar.

Un grafo es bipartito cuando existen $V_1, V_2 \subset V$ que cumplen que son disjuntos y que $V_1 \cup V_2 = V$, tales que todas las aristas del grafo son entre algún elemento de V_1 y algún elemento de V_2 . Por lo tanto, no hay aristas entre nodos del mismo conjunto.

Una forma de determinar si un grafo es bipartito es realizar una recorrida BFS, y determinar si existen dos nodos dentro de una misma capa que estén conectados entre sí, ya que si están en la misma capa (supongamos capa L_j), entonces ambos están a una distancia $j - i$ de un nodo z perteneciente a la capa L_i , y luego como tienen una arista entre sí, se forma un ciclo de largo $2(j - i) + 1$ que es impar.

- DAG (Directed Acyclic Graph) y Orden Topológico

Decimos que un grafo es un DAG si el grafo es dirigido y no contiene ciclos. A su vez, si un grafo es un DAG entonces podemos ordenar los nodos en una lista, de forma que ningún nodo tenga aristas salientes hacia un nodo anterior en el orden.

Dado un grafo dirigido podemos **detectar si dicho grafo contiene un ciclo** si utilizamos el algoritmo DFS, esto sucede cuando explorando los nodos adyacentes de x vemos que un nodo y ya fue marcado como explorado. Luego, iterando hacia atrás sobre el arreglo de padres hasta encontrar a y podemos reconstruir el ciclo.

Para construir un **orden topológico a partir de un DAG**, lo que podemos hacer es una recorrida con BFS contando para cada nodo cuantas aristas incidentes tiene y si no tiene, agregándolo a una lista L . Luego, mientras L no es vacía podemos ir sacando de L un elemento y agregándolos al final del orden OT . Cada vez que sacamos un elemento de la lista, debemos actualizar cuántas aristas incidentes le quedan a los nodos adyacentes (borrado lógico de aristas), y si alguno no tiene aristas incidentes, lo agregamos a L . De esta forma, cuando L queda vacía tendremos un orden topológico en OT .

4 Greedy

Decimos que un algoritmo es Greedy si construye una solución en pequeños pasos, tomando decisiones sobre aspectos locales, buscando optimizar algún criterio global subyacente. Se pueden diseñar distintos algoritmos que procuren optimizar un mismo aspecto de un problema, y que sin embargo, las decisiones que se toman para construir dicha solución sean diferentes.

4.1 Stays Ahead - Exchange Argument

Stays Ahead y Exchange Argument son dos métodos que permiten probar que un algoritmo llega a una solución óptima.

Stays Ahead, se centra en mostrar que la solución obtenida por el algoritmo es al menos tan óptima como una solución óptima hipotética paso a paso, y por lo tanto, se concluye que la solución alcanzada por el algoritmo es óptima.

Exchange Argument, en cambio, asume que se tiene una solución óptima hipotética y busca transformarla gradualmente de forma que se mantenga la optimalidad, hasta llegar a la solución obtenida por el algoritmo. Y como las transformaciones mantienen la optimalidad de la solución, entonces al llegar a nuestra solución, nuestra solución es óptima.

Si bien ambos métodos pueden aplicarse en distintos ámbitos, la aplicación del Stay Ahead resulta más natural cuando a priori desconocemos si la cantidad de elementos de la solución óptima es igual que la de nuestra solución.

Mientras que el Exchange Argument, resulta más natural su aplicación cuando sabemos que el cardinal de nuestra solución y de la óptima es el mismo, y que la única diferencia es el orden de dichos elementos.

4.2 Problemas representativos

4.2.1 Planificación de Intervalos

El problema consiste en un conjunto $1, 2, \dots, n$ de solicitudes, donde la i -ésima solicitud es aquella que corresponde al intervalo que comienza en $s(i)$ y termina en $f(i)$. El objetivo es conseguir conjunto de solicitudes compatibles, es decir, que los intervalos del conjunto no se solapen entre sí. Y también queremos que ese conjunto sea lo más grande posible.

Para generar un algoritmo Greedy que resuelva el problema, primero se debe ordenar los intervalos en función de algún criterio local, en este caso, podría ser ordenar en orden creciente según $s(i)$, o según $f(i) - s(i)$, y luego irlos eligiendo en ese orden y descartando los que se solapan. Sin embargo, esos criterios locales fallan en conseguir una solución óptima, en su lugar un criterio que funciona es ordenar los intervalos en orden creciente según $f(i)$, el algoritmo es el siguiente:

Interval-Scheduling(R)

```
1: Consideramos a  $R$  el conjunto de solicitudes y lo ordenamos de forma creciente según  $f(i)$ .
2:  $A = \emptyset$ 
3: while  $R \neq \emptyset$  do
4:   Elijo el primer elemento de  $R$  y lo agrego a  $A$ .
5:   Elimino de  $R$  dicho elemento y todos los que se solapan con él.
6: end while
   return  $A$ 
```

Primero que nada podemos ver que el conjunto A es consistente ya que elegimos los intervalos para que

no se solapan entre sí.

(Demostración Stays Ahead)

Para probar que la solución A es óptima, supondremos que conocemos una solución óptima $\mathcal{O}$, y buscaremos probar que $|A| = |\mathcal{O}|$. Entonces, sean $A = \{i_1, \dots, i_k\}$ y $\mathcal{O} = \{j_1, \dots, j_m\}$, queremos probar que $k = m$.

Luego, por cómo elegimos los intervalos para A , sabemos que se cumple que $f(i_1) \leq f(j_1)$, por lo tanto, por inducción probamos que para todos los índices $r \leq k$ se cumple que $f(i_r) \leq f(j_r)$.

Luego, suponiendo que A no es óptimo, entonces $m > k$. Si suponemos $r = k$, tenemos que $f(i_k) \leq f(j_k)$, pero entonces existe otro intervalo j_{k+1} que no se solapa con j_k y en especial, tampoco se solapa con i_k , por lo tanto es compatible y no fue eliminado del conjunto R , entonces tenemos una contradicción.

Se concluye que el conjunto A es óptimo, ya que $m = k$.

4.2.2 Minimizar tardanza

El problema consiste nuevamente en un conjunto de n solicitudes para la utilización de un recurso durante un intervalo de tiempo, a diferencia del problema anterior, las solicitudes no tienen un tiempo de comienzo y fin, sino que tienen una duración t_i y una fecha límite d_i . Decimos que hay una tardanza si una solicitud i termina en el tiempo $f(i)$, y la tardanza es $l_i = f(i) - d_i$, siendo $f(i) = s(i) + t_i$.

El criterio Greedy para resolver este problema consiste en ordenar las solicitudes en orden creciente según la fecha límite, y tomar las solicitudes en ese orden, a nuestra solución le llamaremos S .

(Demostración Argument Exchange)

Para probar que este criterio nos da una solución óptima, utilizamos el método del Exchange Argument, suponemos que conocemos una solución óptima $\mathcal{O}$ tal que $|\mathcal{O}| = |S|$.

Si $S = \mathcal{O}$, entonces S es óptima de forma trivial. Sino, entonces en $\mathcal{O}$ existen inversiones respecto del orden definido para S . En particular, existen dos solicitudes i, j consecutivas tales que $i < j$ y $d_j \leq d_i$.

Si deshacemos esa inversión o bien la tardanza se mantiene igual o se minimiza, ya que estamos aprovechando que $d_j \leq d_i$. Luego, como la cantidad de inversiones es finita, si las vamos deshaciendo de a una, como la solución no empeora, al final llegamos nuestra solución S que es tan óptima como $\mathcal{O}$.

4.2.3 Camino más corto en un grafo

El problema que consideremos es dado un grafo $G = (V, E)$ donde las aristas tienen un costo l_e asociado al largo de la arista, por lo tanto, se cumple que $l_e \geq 0$ tenemos dos nodos $s, t \in V$ donde s es el origen y t es el destino, y queremos hallar el camino $s - t$ de costo mínimo.

IMPORTANTE: El costo de las aristas no puede ser negativo, sino Dijkstra puede no llegar a la solución correcta.

Este problema se resuelve con el algoritmo de Dijkstra:

Dijkstra(G,s)

```
1: Inicializo el conjunto  $S = \{s\}$ .
2:  $d(s) = 0$ .
3: while  $S \neq V$  do
4:   Considero todas las aristas  $e = (u, v)$  tales que  $u \in S$  y  $v \notin S$ .
5:   Elijo la arista  $e$  que minimiza  $d(u) + l_e$ .
6:   Agrego nodo  $v$  de dicha arista a  $S$ .
7:    $d(v) = d(u) + l_e$ .
8:    $prev[v] = u$  ▷ Esto permite reconstruir el camino.
9: end while
   return d, prev
```

Prueba de correctitud de Dijkstra

Demostramos esto mediante la inducción en el tamaño de S . El caso $|S| = 1$ es fácil, ya que entonces tenemos $S = \{s\}$ y $d(s) = 0$. Supongamos que la afirmación es cierta cuando $|S| = k$ para algún valor de $k \geq 1$, ahora hacemos crecer S a tamaño $k + 1$ agregando el nodo v . Consideremos la arista (u, v) en el camino final P_v .

Por hipótesis de inducción, P_u es el camino más corto de s a u para cada $u \in S$. Ahora consideremos cualquier otro camino P desde s a v , queremos mostrar que es al menos tan largo como P_v . Para llegar a v , este camino P debe salir del conjunto S en algún lugar, consideremos y el primer nodo en P que no está en S , y $x \in S$ el nodo justo antes de y .

La clave de la prueba es muy simple: P no puede ser más corto que P_v porque ya es al menos tan largo como P_v en el momento en que ha dejado el conjunto S . De hecho, en la iteración $k + 1$, el algoritmo de Dijkstra debe haber considerado agregar el nodo y al conjunto S a través de la arista (x, y) y rechazó esta opción a favor de agregar v . Esto significa que no hay un camino desde s hasta y a través de x que sea más corto que P_v . Pero el subcamino de P hasta y es ese camino, y por lo tanto es al menos tan largo como P_v . Dado que los pesos de las aristas son no negativos, el camino completo P es al menos tan largo como P_v también.

Esta es una prueba completa; también se puede especificar el argumento en el párrafo anterior utilizando las siguientes desigualdades. Sea P' el subcamino de P desde s hasta x . Como $x \in S$, sabemos por la hipótesis de inducción que P_x es un camino más corto desde s hasta x (de longitud $d(x)$), y por lo tanto $l(P') \geq l(P_x) = d(x)$. Por lo tanto, el subcamino de P hasta el nodo y tiene una longitud de $l(P') + l(x, y) \geq d(x) + l(x, y) \geq d'(y)$, y el camino completo P es al menos tan largo como este subcamino. Finalmente, ya que el algoritmo de Dijkstra seleccionó a v en esta iteración, sabemos que $d'(y) \geq d'(v) = l(P_v)$. Combinando estas desigualdades se muestra que $l(P) \geq l(P') + l(x, y) \geq l(P_v)$. Este es una prueba completa.

Implementación de Dijkstra

- Se colocan los nodos V en una cola de prioridad con $d_c(v)$ como la clave para $v \in V$.
- Para seleccionar el nodo v que debe ser agregado a S , se utiliza la operación ExtractMin.
- Para actualizar las claves, se considera una iteración en la que el nodo v es agregado a S .
- Si (v, w) no es una arista, entonces no se tiene que hacer nada.
- Si $e_c = (v, w) \in E$, entonces el nuevo valor para la clave es $\min(d_c(w), d(v) + \omega_e)$.

- Si $d_c(w) > d(v) + \omega_e$ entonces se utiliza la operación ChangeKey para disminuir la clave de w de manera apropiada.
- La operación ChangeKey solo ocurre una vez por arista.

Utilizando una cola de prioridad el algoritmo de Dijkstra puede ser implementado en $O(m \log(n))$.

4.2.4 Árbol de recubrimiento mínimo

El problema del árbol de recubrimiento mínimo consiste en que dado un grafo conexo $G = (V, E)$ donde sus aristas tienen un costo $c_e \geq 0$. Luego, el objetivo es encontrar un subconjunto $T \subseteq E$, tal que el grafo $G = (V, T)$ sea conexo y la suma $\sum_{e \in T} c_e$ sea mínima.

Una observación básica es que el grafo $G = (V, T)$ debe ser un árbol, esto es simple de ver ya que el grafo debe ser conexo, a su vez no puede contener ciclos, ya que podríamos quitar una de las aristas del ciclo y el grafo permanecería conexo, en particular, podríamos eliminar la de máximo costo, y por lo tanto conseguiríamos un grafo conexo de menor costo que el inicial.

Kruskal(G)

- 1: $T = \emptyset$.
 - 2: Ordeno todas las aristas en orden creciente según su costo.
 - 3: Agrego las aristas a T en ese orden siempre que no genere se genere un ciclo.
 - 4: Si una arista genera un ciclo la descarto.
- return** T

Prim(G)

- 1: $S = \{s\}$.
 - 2: $T = \emptyset$.
 - 3: **while** ($S \neq V$) **do**
 - 4: Miro todas las aristas $e = (u, v)$, tal que $u \in S$ y $v \notin S$.
 - 5: Elijo la arista e de costo mínimo y la agrego a T , agrego v a S .
 - 6: **end while**
- return** T

Reverse-Delete(G)

- 1: $T = E$.
 - 2: Ordenamos T en forma decreciente según los costos de las aristas.
 - 3: Eliminamos las aristas de mayor costo siempre que no dejen el grafo inconexo.
- return** T .

Propiedad de Corte *Asumiendo que los costos de todas las aristas son distintos entre sí. Sea S un subconjunto de nodos tal que no es vacío ni igual a V , y sea la arista $e = (u, v)$ la arista de menor costo con un extremo en S y el otro en $V - S$. Entonces cualquier árbol de recubrimiento mínimo contiene la arista e .*

Supongamos por absurdo que existe un árbol de recubrimiento mínimo T' que no incluye la arista e , si no la incluye y e era la única arista entre S y $V - S$ entonces es falso que T' sea un árbol de recubrimiento ya que no es conexo. Por lo tanto, supongamos que existe otra arista e' que une las componentes S y $V - S$, tal que el costo de $c_{e'} > c_e$.

Luego, si consideramos el árbol de recubrimiento $T' = T - \{e\} \cup \{e'\}$, ambos son árboles de recubrimiento, sin embargo el costo de T es claramente menor que el de T' , por lo tanto, llegamos a un absurdo por suponer que T' era árbol de recubrimiento mínimo.

Se concluye entonces que e debe pertenecer a cualquier árbol de cubrimiento mínimo.

Propiedad del ciclo *Asumiendo que los costos de las aristas de G son todos diferentes entre sí. Sea C un ciclo en G , y sea $e = (u, v)$ la arista de mayor costo en C . Entonces e no pertenece a ningún árbol de recubrimiento mínimo de G .*

Sea T un árbol de recubrimiento que contiene la arista e . Si eliminamos la arista e , entonces nos quedan dos componentes conexas: S que contiene a u y $V - S$ que contiene a v .

Ahora, como e pertenece a un ciclo, entonces existe un arista $e' \neq e$ en el ciclo, tal que tiene un extremo en cada componente. Si ahora, construimos $T' = T - \{e\} \cup \{e'\}$, como se cumple que $c_e > c_{e'}$, entonces el costo de T' es menor que el de T , por lo tanto, un árbol de recubrimiento mínimo no puede contener a e .

Se concluye que la arista más costosa de un ciclo no puede pertenecer a un árbol de recubrimiento.

5 Divide y Vencerás

La técnica de Divide y Vencerás es un enfoque para resolver problemas mediante la división del problema en subproblemas más pequeños y manejables, y luego combinando las soluciones de los subproblemas para obtener una solución al problema original. Es una técnica recursiva y se basa en tres pasos:

1. Dividir el problema en subproblemas: se divide el problema en subproblemas más pequeños que sean manejables y similares al problema original.
2. Resolver los subproblemas: se resuelven los subproblemas de manera independiente.
3. Combinar las soluciones de los subproblemas: se combinan las soluciones de los subproblemas para obtener una solución al problema original.

5.1 Recurrencias

Como todos los problemas que pueden resolverse con Divide y Vencerás consiste en resolver una recurrencia, entonces será útil conocer algunas técnicas de resolución.

Desenrollar la recurrencia

1. Desarrollamos los primeros niveles de la recursión.
2. Tratamos de identificar un patrón, sobre cómo crece la cantidad de subproblemas en cada nivel de la recursión.
3. Por último, sumamos sobre todos los niveles de la recursión.

Sustituir en la recurrencia Si por otro lado, tenemos una idea de qué función acota a $T(n)$, entonces podemos introducir dicha suposición y ver si cumple con la recurrencia.

Por ejemplo, si creemos que $T(n) = O(\log(n))$, probamos sustituir $T(n)$ con $c \cdot \log_2(n)$ tal que $c > 0$.

Sustitución parcial Este tipo de sustitución es similar a la anterior en el sentido que estamos intentando adivinar la cota, pero es útil cuando no sabemos qué valor debería tomar la constante o la base del logaritmo, la idea es utilizar parámetros genéricos, sustituir en la recursión y desarrollar para tratar de determinar qué valores deberían tomar en caso que existan.

5.2 Problemas representativos

5.2.1 MergeSort

El algoritmo MergeSort es un algoritmo de ordenamiento, que dada una lista de números divide la lista "al medio" y ordena cada parte por separado de forma recursiva, para luego combinar las partes de forma ordenada en tiempo lineal. Por lo tanto, si denotamos el tiempo que requiere el algoritmo como $T(n)$, vemos que cumple la siguiente recurrencia:

$$T(n) \leq 2T(n/2) + cn$$

$$T(2) \leq c,$$

para alguna constante $c > 0$ y con $n \geq 2$.

Si desenrollamos la recurrencia, vemos el siguiente patrón:

Nivel 0 : cn

Nivel 1: $2(cn/2) = cn$

Nivel 2: $4(cn/4) = cn$

$\vdots$

Nivel j : $2^j(cn/2^j) = cn$

Al desarrollarlo se puede visualizar como un árbol binario, por lo tanto, como la altura de un árbol binario es $\log(n)$, y como en cada nivel se realizan cn operaciones, entonces se cumple que $T(n)$ está acotado superiormente por $O(n\log(n))$ cuando $n > 1$.

Ordeno(L)

```
1: if  $|L| \leq 1$  then
2:   Está ordenado.
3: else
4:    $L_1 =$  Primeros  $\lceil \frac{n}{2} \rceil$  elementos de L.
5:    $L_2 =$  El resto de los elementos de L.
6:    $L_1 = \text{Ordeno}(L_1)$ 
7:    $L_2 = \text{Ordeno}(L_2)$ 
8:    $Res = \text{Combino}(L_1, L_2)$ 
9: end if return Res
```

Combino(L_1, L_2)

```
1: Sean  $i, j$  punteros que apuntan al inicio de  $L_1$  y  $L_2$  respectivamente.
2: Sea  $Res$  una lista vacía.
3: while (no me voy de rango) do
4:   if ( $L_1[i] < L_2[j]$ ) then
5:     Agrego  $L_1[i]$  al final de  $Res$ .
6:     Aumento  $i$ .
7:   else
8:     Agrego  $L_2[j]$  al final de  $Res$ .
9:     Aumento  $j$ .
10:  end if
11: end while
    return Res
```

5.3 Conteo de inversiones

El problema de conteo de inversiones consiste en determinar la similitud de dos listas con cierto, que pueden representar preferencias respecto de algún tema.

Para esto, a una de las listas se le asignan etiquetas a sus n elementos de la forma $a_1, a_2, \dots, a_n$, tal que $a_1 < a_2 < \dots < a_n$. Luego, decimos que en la otra lista existe una inversión si existen dos índices $i < j$, tales que en dicha lista $a_j < a_i$.

Para este problema es útil notar que si dividimos el problema en dos mitades y ordenamos cada mitad, no es necesario hacer todas las comparaciones, sino que podemos ahorrarnos muchas comparaciones, lo que redundará en un mejor tiempo de ejecución.

Uno-Cuento

```
1: Inicializo un puntero Actual al frente de cada lista.
2:  $inv = 0$ .
3:  $Res = \emptyset$ 
4: while Ambas listas no son vacías do
5:   Sean  $a_i$  y  $b_j$  los elementos a puntados por el puntero Actual.
6:   Agrego el elemento más pequeño a Res.
7:   if  $b_j$  es el elemento pequeño then
8:     Incremento  $inv$  por el la cantidad de elementos restantes en A.
9:   end if
10:  Avanzo el puntero Actual en la lista de la cual se eligió el elemento.
11: end while
    return  $Res, inv$ 
```

Ordeno-Cuento

```
1: if  $|L| = 1$  then
2:   No hay inversiones
3: end if
4: Divido la lista en dos mitades.
5: A contiene los primeros  $\lceil n/2 \rceil$  elementos
6: B contiene el resto de los elementos
7:  $i_A = \text{Ordeno\&Cuento}(A)$ .
8:  $i_B = \text{Ordeno\&Cuento}(B)$ .
9:  $i_{A,B} = \text{Uno\&Cuento}(A,B)$ .
    return  $i_A + i_B + i_{A,B}$ 
```

6 Programación Dinámica

La Programación Dinámica es una técnica de diseño de algoritmos, que consiste en explorar todo el espacio de posibles soluciones de forma implícita, mediante la descomposición cuidadosa de un problema en subproblemas más pequeños, y luego construir una solución a subproblemas cada vez más grandes hasta llegar al problema original. En cierta forma, podemos ver que la programación opera peligrosamente cerca de la búsqueda por fuerza bruta, trabajando a través del conjunto de posibles soluciones de tamaño exponencial, sin embargo, lo hace sin examinar todo de forma explícita.

A diferencia de la técnica Divide y Vencerás, la división en subproblemas no es independiente, sino que los subproblemas se solapan entre sí.

Las características de los problemas que pueden ser resueltos eficientemente con Programación Dinámica, se resumen a lo siguiente:

1. Sólo hay una cantidad polinomial de subproblemas.
2. La solución al problema original puede computarse fácilmente a partir de las soluciones a los subproblemas.
3. Hay un ordenamiento natural de "más chico" a "más grande", con una recurrencia fácil de calcular que permite determinar la solución a un subproblema en base a las soluciones de alguna cantidad de subproblemas más pequeños.

6.1 Memoización e iteración

Las soluciones que utilizan la técnica de Programación Dinámica implican descubrir una recursión, y resolverla. Sin embargo, si esto no se implementa de una forma adecuada puede implicar realizar los mismos cálculos reiteradas veces, esto puede hacer que el tiempo de ejecución de un algoritmo sea exponencial.

Como no queremos eso, será útil tener una tabla donde almacenar resultados intermedios que representan los valores óptimos de los subproblemas, de esta forma sólo se calcularán una única vez y luego si los requerimos sólo será necesario consultar dicha tabla en $O(1)$. A esta técnica se le denomina *Memoización*.

Otro detalle importante a la hora de obtener los valores óptimos, es que nosotros conocemos los pasos base y cómo se forma el siguiente valor óptimo, por lo tanto, no es necesario utilizar recursión, podemos simplemente iterar sobre la tabla construyendo los valores a partir de los pasos anteriores. Veremos además, que si iteramos en un sentido para calcular los valores óptimos de los subproblemas, luego deberemos iterar en el otro sentido, desandando los pasos que dimos para poder hallar la solución optimiza un cierto parámetro.

6.2 Problemas representativos

6.2.1 Planificación de Intervalos Ponderados

El problema consiste en dado un conjunto de n intervalos $I = (s_1, f_1, w_1), \dots, (s_n, f_n, w_n)$ que representan trabajos y sus respectivas pagas, donde s_i y f_i indican el tiempo de comienzo y fin de dicho trabajo, mientras que w_i es la paga que se recibe al realizar el trabajo. El objetivo es conseguir un subconjunto $S \subseteq I$ tal que ningún par de intervalos de S se solapen, y buscando maximizar la ganancia.

Si bien se parece al problema que resolvimos con un algoritmo Greedy, en este caso aquel algoritmo no funciona.

Para resolver este problema utilizando Programación Dinámica, primero ordenamos los intervalos en orden creciente respecto a la finalización. Y definimos la notación $p(\cdot)$, que dado un índice j que representa

al j -ésimo intervalo en el orden, $p(j)$ representa el mayor índice i del del orden tal que $i < j$ y no se solapan entre sí. También consideramos la siguiente notación, $OPT(j)$ representa la máxima ganancia que se puede obtener de considerar desde el primer intervalo hasta el j -ésimo.

Para poder resolver este problema, debemos notar dos cosas:

1. El intervalo n , puede formar parte del conjunto solución o no. En caso que pertenezca, nuestra solución tiene valor $w_n + OPT(p(n))$, mientras que si no pertenece nuestra solución tiene valor $OPT(n - 1)$.
2. Si decidimos no considerar ningún trabajo, no obtenemos ninguna ganancia, entonces $OPT(0) = 0$.

Por lo tanto, vemos que se forma la siguiente recurrencia:

$$OPT(0) = 0$$

$$OPT(n) = \max\{w_n + OPT(p(n)), OPT(n - 1)\}$$

El algoritmo para obtener los valores óptimos es:

Computo-OPT

```

1:  $OPT[0] = 0$ 
2: for  $i = 1$  hasta  $n$  do
3:    $OPT[i] = \max\{w_i + OPT[p(i)], OPT[i - 1]\}$ 
4: end for
   return  $OPT[n]$ 

```

Ahora, teniendo dicha tabla podemos construir el subconjunto solución del problema.

Encuentro-Solución

```

1:  $S = \emptyset$ 
2:  $i = n$ 
3: while  $i > 0$  do
4:   if  $(OPT[i] = w_i + OPT[p(i)])$  then
5:     Agrego  $i$  al  $S$ .
6:      $i = p(i)$ 
7:   else
8:      $i = i - 1$ 
9:   end if
10: end while
    return  $S$ 

```

6.2.2 Mínimos Cuadrados Segmentados

En problema anterior la recurrencia se basaba fundamentalmente en decisiones binarias, básicamente, un intervalo podía pertenecer a la solución óptima o no. En este problema sin embargo, la recurrencia implicará en cada paso una cantidad polinomial de posibilidades a considerar para la estructura de la solución óptima. A este tipo de recurrencia se le denomina "*Multi-Way Choices*".

El problema consiste en un conjunto de datos experimentales que representan n puntos en el plano, $P = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, y suponemos que $x_1 < x_2 < \dots < x_n$. Dada una recta L , de la

forma $y = ax + b$, decimos que el error de aproximar los puntos de P mediante la recta L es :

$$Error(L, P) = \sum_{i=1}^n (i_i - ax_i - b)^2$$

El objetivo es aproximar los puntos mediante rectas tratando de minimizar el error, pero a la vez tratando de minimizar la cantidad de rectas utilizadas. Entonces, la idea es agrupar puntos de consecutiva que van a estar aproximados por una misma recta, el objetivo entonces es determinar dónde se deben cortar las secuencias de puntos.

Para esto agregamos la notación $p_i = (x_i, y_i)$ para denotar los puntos. Y notamos que las secuencias de puntos que estamos buscando son en realidad subconjuntos disjuntos de P , tales que las x-coordenadas en dichos subconjuntos son

Para poder contemplar el error que se genera al aproximar por una recta y también controlar qué tantas rectas estamos dispuestos a aceptar, utilizamos una penalización que es la suma de lo siguiente:

- El número de segmentos en los cuales particionamos P , multiplicado por una constante $C > 0$.
- Para cada segmento, el valor del error de la recta óptima por dicho segmento.

Entonces, buscaremos una partición de penalización mínima.

Ahora, para tratar de generar un algoritmo, lo primero es tratar de ver ciertos detalles de la estructura del problema que pueden ser útiles para lograr una recurrencia:

- Estamos buscando una partición de n puntos.
- El último punto p_n pertenece a un único segmento en la solución óptima, y ese segmento empieza un punto p_i anterior. Por lo tanto, si conocemos el último segmento $p_i, \dots, p_n$, podemos dejar de considerarlo para hallar los demás segmentos en $p_1, \dots, p_{i-1}$.
- Si denotamos $OPT(i)$ a la penalización de la solución óptima para los primeros i puntos, y denotamos $e_{i,j}$ como el error mínimo de aproximar los puntos $p_i, \dots, p_j$ por una recta.
- Definimos $OPT(0) = 0$ (Si no hay puntos, no hay penalización).

Sabiendo esto podemos plantear la siguiente recurrencia:

$$\begin{aligned} OPT(0) &= 0 \\ OPT(j) &= \min_{1 \leq i \leq j} \{e_{i,j} + C + OPT(i-1)\}. \end{aligned}$$

Luego, el algoritmo es el siguiente:

Mínimos-Cuadrados-Segmentados(n)

```

1: Defino un array  $OPT[0, \dots, n]$ 
2:  $OPT[0] = 0$ .
3: for (cada par  $i \leq j$  do
4:   Calculo  $e_{i,j}$  para el segmento  $p_i, \dots, p_j$ 
5: end for
6: for ( $j = 1 \dots n$ ) do
7:    $OPT(j) = \min_{1 \leq i \leq j} \{e_{i,j} + C + OPT(i-1)\}$ 
8: end for
   return  $OPT[n]$ 

```

Encuentro-Solución($OPT[.]$, j)

```

1: if ( $j = 0$ ) then
2:   No devuelvo nada
3: else
4:   Encuentro el  $i$  que minimiza  $e_{i,j} + C + OPT[i-1]$ 
5: end if
   return El segmento  $p_i, \dots, p_j$  y Encuentro-Solución( $i-1$ )

```

Algunas puntualizaciones respecto de los algoritmos, es que los $e_{i,j}$ pueden ser calculados en tiempo $O(n^3)$ ya que a lo sumo hay n^2 pares (i, j) y por que calcular cada $e_{i,j}$ implica utilizar la fórmula de la suma de los errores al cuadrado, que requiere $O(n)$ operaciones.

Una vez calculados los errores, el resto del algoritmo requiere un tiempo $O(n^2)$.

6.2.3 Problema del Mochilero

Dado un conjunto de n elementos $I = \{1, \dots, n\}$, tal que cada uno tiene un peso no negativo w_i y un valor v_i , a su vez, también contamos con un valor W .

El Problema del Mochilero consiste en conseguir un subconjunto $S \subseteq I$, tal que $\sum_{i \in S} w_i \leq W$ y a su vez se maximice el valor total del conjunto, sujeto a la restricción anterior.

Un caso particular de este problema es el Problema de la Suma de Subconjuntos, en este problema $v_i = 1$ para todo $i = 1, \dots, n$. Entonces, este problema se traduce a encontrar un conjunto con la máxima cantidad de elementos sin que se sobrepase la cota W .

Ahora, para tratar de construir una recurrencia para el Problema de Suma de Subconjuntos, notamos algunos detalles:

- Debemos utilizar una variable adicional que refleje cuál es la capacidad disponible, ya que es la única forma que tenemos para descartar elementos de la consideración.
- Definimos $OPT(i, w)$ de forma que denote el valor de la solución óptima para el subconjunto $\{1, \dots, i\}$ con una capacidad máxima permitida W , $OPT(i, W) = \max \sum_{j \in S} w_j$.
- Si la capacidad $W = 0$, entonces no se puede elegir ningún elemento, $OPT(i, 0) = 0 \forall i \in I$.
- Si no elegimos ningún elemento, entonces $OPT(0, w) = 0 \ 0 \leq w \leq W$.
- Si $i \in S$, entonces $OPT(i, W) = w_n + OPT(i-1, W - w_n)$
- Si $i \notin S$, entonces $OPT(i, W) = OPT(i-1, W)$

La recurrencia queda de la siguiente manera:

$$OPT(0, w) = OPT(i, 0) = 0 \quad \forall i \quad \forall w$$

$$OPT(i, w) = \max\{w_n + OPT(i-1, W - w_n), OPT(i-1, W)\}$$

Subset-Sum(n, W)

```

1: Defino  $OPT[0...n][0...W]$ 
2: Inicializo  $OPT[0, w] = 0$  para todo  $w = 0, \dots, W$ 
3: for  $i = 1 \dots n$  do
4:   for  $w = 1, \dots, W$  do
5:      $OPT(i, w) = \max\{w_n + OPT(i-1, W - w_n), OPT(i-1, W)\}$ 
6:   end for
7: end for
   return  $OPT[n, W]$ 

```

Este algoritmo sirve para resolver el problema de Subset-Sum, que es un caso particular del Problema del Mochilero, cuando los $v_i = 1$. Para poder adaptarlo, basta con ver que la recurrencia nos queda $OPT(i, w) = \max\{v_n + OPT(i-1, W - w_n), OPT(i-1, W)\}$, el resto es igual.

Encuentro-Solucion($OPT[.]$, n, W)

```

1:  $S = \emptyset$ 
2:  $w = W$ 
3: for  $j = n \dots 0$  do
4:   if  $(OPT[j, w] = w_j + OPT(j-1, W - w_j))$  then  $w = w - w_j$  Agrego  $w_j$  a  $s$ 
5:   end if
6: end for
   return  $S$ 

```

Podemos ver claramente que el tiempo de ejecución del algoritmo Subconjuntos-Suma es $O(nW)$ debido a que implica llenar las entradas de una matriz de $(n+1) \times (W+1)$. Lo mismo sucede para el algoritmo del Problema del Mochilero.

Un detalle importante es que este algoritmo no es polinomial respecto de n , sino que es polinomial respecto a n y W , a este tipo de algoritmos les decimos *pseudo-polinomiales*, ya que si bien son razonablemente eficientes cuando los w_i son pequeños, cuando son valores más grandes se vuelve impracticable.

6.2.4 RNA Estructura Secundaria

En el anterior problema si bien agregamos una nueva variable, cuando hablábamos de un subproblema eran de la forma $\{1, \dots, j\}$ con j variando, sin embargo, algo que será útil es agregar una variable i , tal que $i \leq j$, que nos permita tener subproblemas de la forma $\{i, i+1, \dots, j\}$.

Para el siguiente problema, consideramos una cadena de ARN que puede ser vista como una secuencia de n símbolos que representan las bases $\{A, C, G, U\}$. Se requiere que los elementos A se emparejan con los u , y los C con los G , y también que cada base a lo sumo se puede emparejar con otra base.

Entonces, la estructura la podemos ver como una cadena $B = \{b_1 b_2 \dots b_n\}$, con $b_i \in \{A, C, G, U\}$. Al emparejamiento de las bases lo denominamos "Estructura Secundaria", y está definido como $S = \{(i, j)\}$ donde $i, j \in \{1, 2, \dots, n\}$ tal que se cumplen las siguientes propiedades:

1. Si $(i, j) \in S$, entonces $j - i > 4$.

2. Los elementos de cada par consisten en pares (A, U) y (C, G) , cualquier orden.
3. S es un emparejamiento, ninguna base aparece en más de un par.
4. Si (i, j) y (k, l) son dos pares en S , entonces no puede pasar que $i < k < j < l$.

Ahora, para poder armar la recurrencia notamos las siguientes cosas:

- Denotamos $OPT(i, j)$ como la máxima cantidad de pares de bases en la sub tira de bases $b_i b_{i+1} \dots b_j$.
- $OPT(i, j) = 0$ si $j - i \leq 4$ y si $i > j$.
- Si j no pertenece a un par, entonces $OPT(i, j) = OPT(i, j - 1)$.
- Si j pertenece a un par, entonces se empareja con un t tal que $i \geq t < j$, por lo tanto, $OPT(i, j) = 1 + OPT(i, t - 1) + OPT(t + 1, j - 1)$.

Luego, la recurrencia es:

$$OPT(i, j) = 0 \text{ si } j - i \leq 4 \text{ y si } i > j.$$

$$OPT(i, j) = \max\{OPT(i, j - 1), 1 + OPT(i, t - 1) + OPT(t + 1, j - 1)\}$$

Estructura-Secundaria-OPT

```

1: Defino  $OPT[n][n]$ 
2: Inicializo  $OPT[i, j] = 0$  cuando  $j - i \leq 4$  y cuando  $i > j$ .
3: for  $k = 5, \dots, n - 1$  do
4:   for  $i = 1, \dots, n - k$  do
5:      $j = i + k$ 
6:      $OPT[i, j] = \max\{OPT(i, j - 1), 1 + OPT(i, t - 1) + OPT(t + 1, j - 1)\}$ 
7:   end for
8: end for
   return  $OPT[1, n]$ 

```

6.2.5 Alineación de secuencias

Supongamos que nos dan las tiras de símbolos X e Y , tales que son de la forma $x_1 x_2 \dots x_m$ y $y_1 y_2 \dots y_n$, respectivamente. Consideramos los conjuntos $\{1, 2, \dots, m\}$ y $\{1, 2, \dots, n\}$ que representan las distintas posiciones de los símbolos en cada tira, consideramos un emparejamiento entre ambos conjuntos.

Decimos que un emparejamiento M de ambos conjuntos es un alineamiento si no hay cruzamiento de pares, es decir, si $(i, j), (i', j') \in M$ y $i < i'$, entonces $j < j'$.

Ahora, para definir una noción de similaridad entre dos tiras X e Y , definimos algunas notaciones:

1. Existe un parámetro $\delta > 0$ que define penalización por dejar un espacio. Para cada posición de X o Y que no están emparejadas entre sí, incurrimos en un costo δ .
2. Para cada par de símbolos p, q , existe un costo por error de discordancia $\alpha_{p,q}$ por haber alineado p con q . Por lo tanto, para cada $(i, j) \in M$ pagamos un costo α_{x_i, y_j} . Asumimos que $\alpha_{p,p} = 0$.
3. El costo total del emparejamiento M es la suma de los costos δ y los α_{x_i, y_j} , y buscamos el emparejamiento de costo mínimo.

Es importante notar lo siguiente para poder construir una recursión. En un alineamiento óptimo M , se cumple al menos una de las siguientes afirmaciones:

- $(m, n) \in M$
- La m -ésima posición de X no está emparejada
- La n -ésima posición de X no está emparejada

Entonces, podemos ver que la recursión es la siguiente:

$$\begin{aligned} OPT(0, j) &= \delta j \quad \forall j \text{ (Esto equivale a desplazar toda la tira } Y) \\ OPT(i, 0) &= \delta i \quad \forall i \text{ (Esto equivale a desplazar toda la tira } X) \\ OPT(i, j) &= \min\{\alpha_{x_i, y_j} + OPT(i-1, j-1), \delta + OPT(i-1, j), \delta + OPT(i, j-1)\} \end{aligned}$$

7 Redes de Flujo

7.1 Definiciones

Nodo fuente: Decimos que un nodo es fuente si en él se genera el tráfico o flujo.

Nodo sumidero: Decimos que un nodo es sumidero si absorbe el flujo.

Flujo: Definimos el flujo como una entidad abstracta, que es generada en los nodos fuente, es transmitido a través de las aristas, y es absorbido por los nodos sumidero.

Red de Flujo: Decimos que un grafo $G = (V, E)$ si cumple lo siguiente:

1. Todas las aristas e del grafo tienen una capacidad $c_e \geq 0$.
2. Hay un único nodo $s \in V$ fuente.
3. Hay un único nodo $t \in V$ sumidero.

Nodos internos: Son aquellos nodos que no son ni fuente ni sumidero.

Flujo s-t: Decimos que una función $f : E \rightarrow \mathcal{R}^+$, donde $f(e)$ representa el flujo que pasa por la arista e , es un flujo s-t si cumple lo siguiente:

1. Para todo nodo interno $x \in V$, el flujo entrante en x es igual al saliente.
2. Para toda arista $e \in E$, $0 \leq f(e) \leq c_e$

Grafo Residual: Dada una red de flujo G , y un flujo f en G , definimos el grafo residual G_f de G respecto a f de la siguiente forma:

- El conjunto de nodos de G_f es igual al de G .
- Para cada arista $e = (u, v)$ en G , si $f(e) < c_e$ entonces en G_f hay una arista (u, v) con capacidad $c_e - f(e)$ (forward).
- Para cada arista $e = (u, v)$ en G , si $f(e) > 0$ entonces en G_f hay una arista (v, u) con capacidad $f(e)$ (backward).

Cuello de botella: Sea P un camino $s - t$ simple en G_f , definimos el cuello de botella en P para el flujo f como la mínima capacidad residual de las aristas del camino.

7.2 Flujo máximo

Dada una red de flujo queremos asignar flujo a las aristas de forma eficiente, con el objetivo de maximizar el flujo total de la red. Para esto utilizamos el algoritmo de Ford-Fulkerson:

Aumentar(f,P)

```
1: Sea  $b$  el cuello de botella del camino  $P$  respecto del flujo  $f$ 
2: for (cada arista  $e = (u, v) \in P$ ) do
3:   if ( $e$  es una arista forward) then
4:     Incrementar  $f(e)$  en  $G$  en  $b$  unidades.
5:   else ( $(u, v)$  es una arista backward, sea  $e' = (v, u)$ )
6:     Decrementamos  $f(e')$  en  $G$  en  $b$  unidades.
7:   end if
8: end for
   return  $f$ 
```

Flujo Máximo(G, G_f)

```
1: Inicialmente  $f(e) = 0 \quad \forall e \in E$ 
2: while (hay un camino  $s - t$  en el grafo residual  $G_f$ ) do
3:   Sea  $P$  un camino simple  $s - t$  en  $G_f$ .
4:    $f' = \text{Aumentar}(f, P)$ 
5:   Actualizar  $f$  para ser  $f'$ 
6:   Actualizar el grafo residual  $G_f$  para ser  $G_{f'}$ 
7: end while
   return  $f$ 
```

El tiempo de ejecución del algoritmo es $O(mC)$, siendo m la cantidad de aristas y C la capacidad de algún corte.

- Sea f un flujo $s - t$, y (A, B) cualquier corte $s - t$. Entonces, $v(f) = f^{OUT}(A) - f^{IN}(A) = f^{IN}(B) - f^{OUT}(B)$.
- Sea f un flujo $s - t$ y (A, B) cualquier corte $s - t$. Entonces, $v(f) \leq c(A, B)$.
- De esto último, se deriva que el valor máximo de un flujo $s - t$ coincide con la mínima capacidad de los cortes $s - t$.

8 Intratabilidad

8.1 Definiciones

- **Reducciones** Dados dos problemas X e Y , decimos que X es al menos tan difícil como Y , si sabiendo cómo resolver X podemos resolver Y .

Esto, lo podemos escribir de forma compacta como $Y \leq_p X$.

Esto significa que si conocemos un algoritmo que resuelve X en tiempo polinomial, entonces, cualquier instancia del problema Y puede resolverse llevando a cabo una cantidad polinomial de pasos computacionales, más una cantidad polinomial de llamadas del algoritmo que resuelve a X .

Por lo tanto, si X puede resolverse en tiempo polinomial, Y también. Y a su vez, si Y no puede resolverse en tiempo polinomial, entonces X tampoco.

Dado que los problemas con los que vamos a trabajar no tienen solución conocida, se utilizará la relación $\leq_p$, para indicar niveles de dificultad entre los problemas.

- **Transitiva de las reducciones** Sean X, Y, Z problemas tales que $X \leq_p Y$ y $Y \leq_p Z$, entonces $X \leq_p Z$.

- La entrada de un problema computacional se codifica como una tira binaria finita s .

- Identificamos un problema de decisión cuando recibe como entrada una tira s y retorna el valor SI o NO . Y decimos que A resuelve el problema X si para cualquier tira s , $A(s) = SI$ si y sólo si $s \in X$. Decimos que A se ejecuta en tiempo polinomial, si existe un polinomio $p(\cdot)$ tal que para toda entrada s , el algoritmo A termina en a lo sumo $O(p(|s|))$ pasos.

- **Certificador eficiente** Decimos que un algoritmo B es un certificador eficiente para un problema X , si dada una instancia del problema codificada en s y un certificado t , que contiene la evidencia de que s es una instancia SI de X , se cumplen las siguientes propiedades:

B tiene tiempo de ejecución polinomial que toma s y t como argumentos.

Existe una función polinomial p tal que para toda tira s , $s \in X$ si y sólo si existe una tira t tal que $|t| \leq p(|s|)$ y $B(s, t) = SI$.

El algoritmo B no resuelve el problema, sólo chequea si $s \in X$ en función de cierta evidencia t .

- **Clase P** Decimos que un problema pertenece a esta clase si existe un algoritmo A que en tiempo polinómico respecto del tamaño de la entrada, resuelve la instancia del problema.

- **Clase NP** Decimos que un problema pertenece a la clase NP si existe un certificador eficiente B .

- $P \subseteq NP$.

- **Clase NP-Completo** Decimos que un problema X pertenece a la clase NP-Completo si:

- $X \in NP$.
- Para todo problema $Y \in NP$, $Y \leq_p X$.

- Esta última definición tiene una gran implicancia, si se pudiera resolver un problema NP-Completo entonces se podrían resolver todos los problemas pertenecientes a la clase NP.

8.2 Ejemplos de problemas NP

- **Independent Set** Dado un grafo $G = (V, E)$, decimos que $S \subseteq V$, es un Independent Set si para cualquier par de nodos del conjunto no hay aristas entre sí.

El problema se formula como: "Dado un grafo G y un número k , ¿ G contiene un conjunto independiente de tamaño al menos k ?"

- **Vertex Cover** Dado un grafo $G = (V, E)$, decimos que $S \subseteq V$, es un Vertex Cover si para todas las aristas del grafo, se cumple que al menos un extremo de la arista se encuentra en el conjunto S .

El problema se formula como: "Dado un grafo G y un número k , ¿ G contiene un Vertex Cover de tamaño a lo sumo k ?"

Si bien, no sabemos resolver de forma eficiente ninguno de los dos problemas, si sabemos que dado un grafo $G = (V, E)$, si S es un Independent Set, entonces $V - S$ es un Vertex Cover. Por lo tanto, $\text{Independent Set} \leq_p \text{Vertex Cover}$ y también $\text{Vertex Cover} \leq_p \text{Independent Set}$.

- **Set Cover** Dado un conjunto U de n elementos, una colección $S_1, \dots, S_m$ de subconjuntos de U y número k , ¿existe una colección de a lo sumo k de esos subconjuntos tales que su unión es igual a U ?

Como el Vertex Cover es un caso particular del Set Cover, cuando los subconjuntos representan el subconjunto de aristas que tienen un extremo en un determinado nodo y el $U = E$, entonces se cumple que $\text{Vertex Cover} \leq_p \text{Set Cover}$.

- **Set Packing** Dado un conjunto U de n elementos, una colección $S_1, \dots, S_m$ de subconjuntos de U , y un número k . ¿Existe una colección de al menos k de esos subconjuntos tal que la intersección entre ellos sea vacía?

- **SAT** Dado un conjunto de n variables booleanas $x_1, \dots, x_n$, se pueden formar términos de la forma x_i o $\bar{x}_i$. A partir de la disyunción de los términos, podemos construir "cláusulas" de la siguiente forma $C = t_1 \vee t_2 \vee \dots \vee t_l$.

El problema SAT consiste en dado un conjunto de cláusulas $C_1, \dots, C_k$ sobre un conjunto de variables $X = \{x_1, \dots, x_n\}$, ¿existe una asignación de valores de verdad para las variables tal que todas las cláusulas se satisfacen a la vez? Pedir que se cumpla cada una por separado, es equivalente a pedir que se satisfaga $C_1 \wedge C_2 \wedge \dots \wedge C_k$.

- **3-SAT** El problema de 3-SAT es un caso particular del problema anterior, para cuando las cláusulas están formadas por disyunción de tres términos. Dado un conjunto de cláusulas $C_1, \dots, C_k$ donde C_i está formada por tres términos, sobre un conjunto de variables $X = \{x_1, \dots, x_n\}$, ¿existe una asignación de valores de verdad para las variables tal que todas las cláusulas se satisfacen a la vez?

Todos estos problemas son NP-Complejos, otros problemas NP-Complejos son: Circuit Satisfiability, Clique, Hamiltonian Cycle, Traveling Salesman, Hamiltonian Path, entre muchos otros.

8.3 Estrategias de pruebas

A continuación se detallará la estrategia básica para probar que un problema X es NP-Completo.

1. Probamos que $X \in NP$.
2. Elegimos un problema Y que sabemos que es NP-Completo.
3. Probamos que $Y \leq_p X$.

(1) Para probar que $X \in NP$, debemos mostrar que existe un algoritmo B que es certificador eficiente. Para esto, debemos considerar una instancia s del problema X , y ver que podemos construir un certificado t de tamaño polinomial respecto del tamaño de la instancia s .

Luego, debemos definir los pasos que sigue el algoritmo $B(s, t)$ para tratar de chequear si s es una instancia SI del problema X , basándonos en la evidencia que contiene t . El tiempo de ejecución de B debe ser polinomial respecto del tamaño de la entrada.

Por último, debemos probar que: $B(s, t)$ responde $SI \iff s$ es una instancia SI de X .

(2) Muchas veces ya nos dan un problema Y con el que debemos trabajar. Sin embargo, en caso que podamos elegirlo, es importante notar que algunos problemas consisten en maximizar un conjunto, como el Independent Set, mientras que otros procuran minimizarlo, como el Vertex Cover. Una buena idea puede ser determinar qué tipo de problema es X , y elegir un Y que coincida en ese sentido, para luego poder hacer una transición más sencilla entre problemas.

(3) Considerar una instancia arbitraria s_Y del problema Y , explicar cómo se podría transformar en una instancia s_X del problema X en tiempo polinomial respecto del tamaño de la instancia.

Y luego, demostrar que: s_Y es una instancia SI de $Y \iff s_x$ es una instancia SI de X .

Por último, como $Y \in NP$ -Completo, y demostramos que $Y \leq_p X$, entonces $X \in NP$ -Completo por transitividad de $\leq_p$.